IMPLEMENTING HIGH-PERFORMANCE COMPLEX MATRIX MULTIPLICATION VIA THE 1M METHOD

FIELD G. VAN ZEE*

Abstract. Almost all efforts to optimize high-performance matrix-matrix multiplication have been focused on the case where matrices contain real elements. The community's collective assumption appears to have been that the techniques and methods developed for the real domain carry over directly to the complex domain. As a result, implementors have mostly overlooked a class of methods that compute complex matrix multiplication using only real matrix products. This is the second in a series of articles that investigate these so-called induced methods. In the previous article, we found that algorithms based on the more generally applicable of the two methods—the 4M method—lead to implementations that, for various reasons, often underperform their real domain counterparts. To overcome these limitations, we derive a superior 1M method for expressing complex matrix multiplication, one which addresses virtually all of the shortcomings inherent in 4M. We show that the method is actually a special case of a larger family of algorithms based on a 2M method, which is generally well-suited for storage formats that organize real and imaginary parts into separate matrices. Implementations are developed within the BLIS framework, and testing on a recent Intel microarchitecture confirms that the 1M method yields performance that is competitive with solutions based on conventionally implemented complex kernels.

Key words. high-performance, complex, matrix, multiplication, microkernel, kernel, BLAS, BLIS, 1m, 2m, 4m, induced, linear algebra, DLA

AMS subject classifications. 65Y04

1. Introduction.

Over the last several decades, matrix multiplication research has resulted in methods and implementations that primarily target the real domain. Recent trends in implementation efforts have condensed virtually all matrix product computation into relatively small *kernels*—building blocks of highly optimized code (typically written in assembly language) upon which more generalized functionality is constructed via various levels of nested loops [23, 6, 4, 22, 3]. Because most effort is focused on the real domain, the complex domain is either left as an unimplemented afterthought—perhaps because the product is merely a proof-of-concept or prototype [6], or because the project primarily targets applications and uses cases that require only real computation [3]—or it is implemented in a manner that mimics the real domain down to the level of the assembly kernel [23, 4, 5].[1] Most modern microarchitectures lack machine instructions for directly computing complex arithmetic on complex numbers, and so when the effort to implement these kernels is undertaken, kernel developers encounter additional programming challenges that do not manifest in the real domain. Specifically, these kernel developers must explicitly orchestrate computation on the real and imaginary components in order to implement multiplication and addition on complex scalars, and they must do so in terms of vector instructions to ensure high performance is achievable.

Pushing the nuances and complexities of complex arithmetic down to the level of the kernel allows the higher-level loop infrastructure within the matrix multiplication

*Oden Institute for Computational Engineering & Sciences, The University of Texas at Austin, Austin, TX (field@cs.utexas.edu)

Funding: This research was partially sponsored by grants from Intel Corporation and the National Science Foundation (Award ACI-1550493). *Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation (NSF).*

[1] Because they exhibit slightly less favorable numerical properties, we exclude Strassen-like efforts from this characterization.

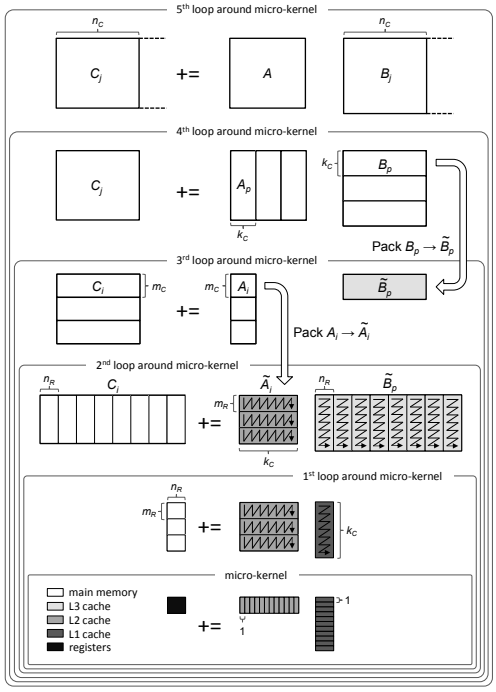

FIG. 1.1. *An illustration of the algorithm for computing high-performance matrix multiplication, taken from [21], which expresses computation in terms of a so-called "block-panel" subproblem.*

to remain largely the same as its real domain counterpart. (See Figure 1.1.) However, this approach also doubles the number of assembly kernels that must be written in order to fully support computation in either domain (real or complex) for the desired floating-point precisions. And while computation in the complex domain may not be of interest to all developers, it is absolutely essential for many fields and applications in part because of complex numbers' unique ability to encode both the phase and magnitude of a wave. Thus, the maintainers of general-purpose matrix libraries—such as those that export the Basic Linear Algebra Subprograms (BLAS) [2]—are typically compelled by their diverse user bases to support complex matrix multiplication despite the implementation and maintenance costs it may impose.

Life would be simpler for matrix library developers if complex matrix multiplication was not necessary. Of course, complex matrix multiplication will always be necessary. But what if complex matrix multiplication *kernels* were found to be unnecessary? To certain actors, particularly matrix library developers, such a finding would carry non-trivial consequence.

The predecessor to the current article investigates whether (and to what degree of effectiveness) real domain matrix multiplication kernels can be repurposed and leveraged toward the implementation of complex matrix multiplication [21]. In that article, the authors develop a new class of algorithms that implement these so-called "induced methods" for matrix multiplication in the complex domain. Instead of relying on an assembly-coded complex kernel, as a conventional implementation would, these algorithms express complex matrix multiplication only in terms of real domain

primitives.[2] The authors first investigate the 4M method, which is merely the matrix analogue of the classic definition of complex scalar multiplication and addition. They show how this method's implementation is facilitated by reordering real and imaginary elements within the internal storage format used when making temporary contiguous "packed" copies of the current matrix blocks. The second[3] approach, the 3M method, benefits from requiring only three (instead of four) matrix products at the cost of an additional three accumulations. This Strassen-like method, which is capable of faster runtimes but exhibits slightly lower numerical accuracy, is implemented by employing a special data format similar to that of 4M, except that it also serves as temporary workspace for storing the results of intermediate computations.[4]

We consider the current article a companion and follow-up to that previous work [21]. Here, we will once again consider a new method for emulating complex matrix multiplication using only real domain building blocks, and we will once again show that a rearrangement of the real and imaginary elements within the packed matrices is key to facilitating an induced method capable of delivering performance that rivals solutions based on the conventional approach.

1.1. Audience. It is also worth emphasizing that this article's primary audience consists of dense linear algebra (DLA) library implemenators and related experts. While this audience is rather narrow, the contributions we present have the potential for wide indirect impact since the community relies heavily on this relatively small group of implementors to develop and maintain DLA kernels for the benefit of the entire high-performance computing (HPC) ecosystem.

1.2. Contributions. This article makes the following contributions:

- It introduces a new induced method—the 1M method—that replaces each complex matrix multiplication with only a single real matrix multiplication. We introduce two algorithms and, as with the previous article, analyze issues germane to their high-performance implementations, including workspace, packing formats, cache behavior, multithreadability, and programming effort. A detailed review shows how 1M avoids virtually all of the challenges observed of the 4M method.
- It promotes code reuse and portability by continuing the previous article's focus on solutions which may be cast in terms of real matrix multiplication kernels. Such solutions have clear implications for developer productivity, as they allow kernel authors to focus their efforts on fewer and simpler kernels.
- It builds on the theme of the BLIS framework as a productivity multiplier [22], further demonstrating how complex matrix multiplication may be implemented with relatively minor modifications to the source code and in such a way that results in immediate instantiation of complex implementations for *all* level-3 BLAS-like operations.
- It demonstrates performance of 1M implementations that is not only superior to the previous effort based on the 4M method but also competitive with

[2] In [21], the authors use the term "primitive" to refer to a functional abstraction that implements a single real matrix multiplication. Such primitives are often not general purpose, and may come with significant prerequisites to facilitate their use.

[3] While this was the first method investigated, the authors of [21] actually choose to present them in reverse order.

[4] Others have exploited the careful design of packing and computational primitives in an effort to improve performance, including in the context of Strassen's algorithm [8, 10, 11, 12], the computation of the K-Nearest Neighbors [24], and tensor contraction [9].

solutions based on complex matrix kernels.
- It serves as a reference guide to the 1M implementations for complex matrix multiplication found within the BLIS framework, which is available to the community under an open source software license.[5]

We believe that these contributions are consequential because the 1M method effectively obviates the previous state-of-the-art established via the 4M method. Furthermore, we believe the thorough treatment of induced methods encompassed by the present article and its predecessor will have lasting archival as well as pedagogical value, to say nothing of the potential impact on developer productivity.

1.3. Notation. In this article, we continue the notation established in [21]. Specifically, we use uppercase Roman letters (e.g. A, B, and C) to refer to matrices, lowercase Roman letters (e.g. x, y, and z) to refer to vectors, and lowercase Greek letters (e.g. χ, ψ, and ζ) to refer to scalars. Subscripts are used typically to denote sub-matrices within a larger matrix (e.g. $A = \left(\begin{array}{c|c|c|c} A_0 & A_1 & \cdots & A_{n-1} \end{array} \right)$) or scalars within a larger matrix or vector.

We make extensive use of superscripts to denote the real and imaginary components of a scalar, vector, or (sub-)matrix. For example, $\alpha^r, \alpha^i \in \mathbb{R}$ denote the real and imaginary parts, respectively, of a scalar $\alpha \in \mathbb{C}$. Similarly, A^r and A^i refer to the real and imaginary parts of a complex matrix A, where A^r and A^i are real matrices with dimensions identical to A. Note that while this notation for real, imaginary, and complex matrices encodes information about content and origin, it does not encode how the matrices are actually stored. We will explicitly address storage details as implementation issues are discussed.

Also, at times we find it useful to refer to the real and imaginary elements of a complex object indistinguishably as *fundamental elements* (or F.E.). We also abbreviate floating-point operations as "flops" and memory operations as "memops". We define the former to be a MULTIPLY or ADD (or SUBTRACT) operation whose operands are fundamental elements and the latter to be a load or store operation on a single fundamental element. These definitions allow for a consistent accounting of complex computation relative to the real domain.

We also discuss cache and register blocksizes that are key features of the matrix multiplication algorithm discussed elsewhere [22, 20, 21]. Unless otherwise noted, blocksizes n_C, m_C, k_C, m_R, and n_R refer to those appropriate for computation in the real domain. Complex domain blocksizes will be denoted with a superscript ^{z}.

This article discusses and references several hypothetical algorithms and functions. Unless otherwise noted, a call to function FUNC that implements $C := C + AB$ appears as $[\ C\ ] := \text{FUNC}(\ A,\ B,\ C\ )$. We will also reference functions that access properties of matrices. For example, $\text{M}(A)$ and $\text{N}(A)$ would return the m and n dimensions of a matrix A, while $\text{RS}(B)$ and $\text{CS}(B)$ would return the row and column strides of B.

2. Background and review.

2.1. Motivation. In [21], the authors list three primary motivating factors behind their effort to seek out methods for inducing complex matrix multiplication via real domain kernels:

- **Productivity.** By inducing complex matrix multiplication from real domain kernels, the number of kernels that must be supported would be halved.

[5] The BLIS framework is available under the so-called "new" or "modified" or "3-clause" BSD license.

This allows the DLA library developers to focus on a smaller and simpler set of real domain kernels. This benefit would manifest most obviously when instantiating BLAS-like functionality on new hardware [20].

- **Portability.** Induced methods avoid dependence on complex domain kernels because they encode the idea of complex matrix product at a higher level. This would naturally allow us to encode such methods portably within a framework such as BLIS [22]. Once integrated into the framework, developers and users would benefit from the immediate availability of complex matrix multiplication implementations whenever real matrix kernels were present.
- **Performance.** Implementations of complex matrix multiplication that rely on real domain kernels would likely inherit the high-performance properties of those kernels. Any improvement to the real kernels would benefit both real and complex domains.

Thus, it is clear that finding a suitable induced method would carry significant benefit to DLA library and kernel developers.

2.2. The 3m and 4m methods. The authors of [21] investigated two general ways of inducing complex matrix multiplication: the 3M method and the 4M method. These methods are then contrasted to the conventional approach, whereby a blocked matrix multiplication algorithm is executed with a complex domain kernel—one that implements complex arithmetic at the scalar level, in assembly language.

The 4M method begins with the classic definition of complex scalar multiplication and addition in terms of real and imaginary components of $\alpha, \beta, \gamma \in \mathbb{C}$:

$$\begin{aligned} \gamma^r &:= \gamma^r + \alpha^r \beta^r - \alpha^i \beta^i \\ \gamma^i &:= \gamma^i + \alpha^i \beta^r + \alpha^r \beta^i \end{aligned} \tag{2.1}$$

We then observe that we can apply such a definition to complex matrices $A \in \mathbb{C}^{m \times k}$, $B \in \mathbb{C}^{k \times n}$, and $C \in \mathbb{C}^{m \times n}$, provided that we can reference the real and imaginary parts as logically separate submatrices:

$$\begin{aligned} C^r &:= C^r + A^r B^r - A^i B^i \\ C^i &:= C^i + A^i B^r + A^r B^i \end{aligned} \tag{2.2}$$

This definition expresses a complex matrix multiplication in terms of four matrix products (hence the name 4M) and four matrix accumulations (i.e., additions or subtractions).

The 3M method relies on a Strassen-like algebraic equivalent of Eq. 2.2:

$$\begin{aligned} C^r &:= C^r + A^r B^r - A^i B^i \\ C^i &:= C^i + \big(A^r + A^i\big)\big(B^r + B^i\big) - A^r B^r - A^i B^i \end{aligned}$$

This re-expression reduces the number of matrix products to three at the expense of increasing the number of accumulations from four to seven. However, when the cost of a matrix product greatly exceeds that of an accumulation, this trade-off can result in a net reduction in computational runtime.

The authors of [21] observe that both methods may be applied to any particular level of a blocked matrix multiplication algorithm, resulting in several algorithms, each exhibiting somewhat different properties. The blocked algorithm used in that article is shown in Figure 1.1 and revisited in Section 2.4 of the present article.

Algorithms that implement the 3M method were found to yield "effective flops per second" performance that not only exceeded that of 4M, but also approached or exceeded the theoretical peak rate of the hardware.[6] Unfortunately, these compelling results come at a cost: the numerical properties of implementations based on 3M are slightly less robust than that of algorithms based on the conventional approach or 4M. And although the author of [7] found that 3M was stable enough for most practical purposes, many applications simply will not be willing to stray from the numerical expectations implicit in conventional matrix multiplication. Thus, going forward, we will turn our attention away from 3M and instead focus on the 4M as the standard reference method against which we will compare.

2.3. Previous findings. For the reader's convenience, we will now summarize the key findings, observations, and other highlights from the previous article regarding algorithms and implementations based on the 4M method [21].

- Since all algorithms in the 4M family execute the same number of flops, the algorithms' relative performance depends entirely on (1) the number of memops executed and (2) the level of cache from which fundamental elements of matrices A and B (or rather, F.E. of the packed copies of these matrices, $\tilde{A}_i$ and $\tilde{B}_p$) are reused[7]. The number of memops is affected only by a halving of certain cache blocksize needed in order to leave cache footprints of $\tilde{A}_i$ and $\tilde{B}_p$ unchanged. The level from which F.E. are reused is determined by the level of the matrix multiplication algorithm to which the 4M method was originally applied. The lower the 4M method is applied, the higher the efficiency of data reuse from and movement through the cache hierarchy.
- The lowest-level application, algorithm 4M_1A, efficiently moves F.E. of A, B, and C from main memory to the L1 cache only once per rank-k_C update, with virtually no excess movement due to incidental cache line proximity, and reuses F.E. from the L1 cache. It relies on a relatively simple packing format in which complex micro-panels are stored with real and imaginary F.E. separated into two consecutive real micro-panels, each with identical register blocksize and k dimensions. Algorithm 4M_1A requires negligible workspace (limited to the storage capacity of the vector register set), is well-suited for multithreading, and is minimally disruptive to the encoding within the BLIS framework. And while algorithm 4M_1B—a slightly higher-level application—very narrowly outperforms 4M_1A by trading away the most optimal cache reuse behavior for an unreduced k_C cache blocksize, 4M_1A proves to be more versatile and can be extended relatively easily to all other level-3 operations.
- The conventional assembly-based approach to complex matrix multiplication can be viewed as a special case of 4M in which F.E. are reused from registers rather than cache. In this way, a conventional implementation embodies the lowest-level application of 4M possible, in which the method is applied to individual scalars (and, typically, then optimally encoded via vector instructions).
- The way complex numbers are stored has a significant effect on performance.

[6] Note that 3M and other Strassen-like algorithms are able to exceed the hardware's theoretical peak performance when measured in *effective* flops per second: that is, the 3M implementation's wall clock time—now shorter because of avoided matrix products—divided into the flop count of a *conventional* algorithm.

[7] Here, the term "reuse" refers to the reuse of F.E. that corresponds to the recurrence of A^r, A^i, B^r, and B^i in Eq. 2.2, not the reuse of whole (complex) elements that naturally occurs in the execution of the matrix multiplication algorithm in Figure 1.1.

Interleaved pair-wise storage of real and imaginary values naturally favors implementations that reuse F.E. from vector registers, as is common with conventional implementations.[8] However, this storage is awkward for algorithms based on 4M (and 3M), largely because it prevents the use of vector instructions for loading and storing F.E. of C^r and C^i[9][10]. The 4M_1A algorithm already suffers from a *quadrupling*[11] of the number of memops on C, in addition to being forced to access these F.E. in a non-contiguous manner. If, however, applications stored complex matrices with real and imaginary parts separated, the penalty paid by 4M (and 3M) would be partially mitigated.

- While observed performance of low-level applications of 4M is decent, far exceeding an unoptimized reference implementation, it not only falls short of a comparable conventional solution based on a complex kernel, it appears to fall short of its real domain "benchmark"—that is, the performance of a similar problem size in the real domain computed by an optimized algorithm using the same real domain kernel. This level of performance may be disappointing for some, even if it achieves 90-95% of what is possible with a complex kernel. The authors conclude that its somewhat attenuated performance would relegate 4M, in practice, to serving mostly as a placeholder, to be used when complex kernels have not yet been written, rather than a competitive replacement that makes complex kernels unnecessary over a longer time horizon.

2.4. Revisiting the matrix multiplication algorithm. In this section, we review a common algorithm for high-performance matrix multiplication on conventional microprocessor architectures. This algorithm was first reported on in [4] and further refined in [22]. Figure 1.1 illustrates the key features of this algorithm.

The current state-of-the-art formulation of the matrix multiplication algorithm consists of six loops, the last of which resides within a micro-kernel that is typically highly optimized for the target hardware. These loops partition the matrix operands using carefully chosen cache (n_C, k_C, and m_C) and register (m_R and n_R) blocksizes that result in submatrices residing favorably at various levels of the cache hierarchy so as to allow data to be reused many times. In addition, submatrices of A and B are copied ("packed") to temporary workspace matrices ($\tilde{A}_i$ and $\tilde{B}_p$, respectively) in such a way that allows the micro-kernel to subsequently access matrix elements contiguously in memory, which improves cache and TLB performance. The cost of this packing is amortized over enough computation that its impact on overall performance is negligible for all but the smallest problems. At the lowest level, within the micro-kernel loop, an $m_R \times 1$ micro-column and a $1 \times n_R$ micro-row are loaded from the current micro-panels of $\tilde{A}_i$ and $\tilde{B}_p$, respectively, so that the outer product of these vectors may be computed to update the corresponding $m_R \times n_R$ submatrix, or micro-tile, of C. The individual floating-point operations that constitute these tiny rank-1 updates are oftentimes executed via vector instructions (if the architecture supports them) in order to maximize utilization of the floating-point unit(s).

[8] The advantages of interleaving data in advance of computation via vector instructions is also discussed by the authors of [1] in the context of performing "batched" level-3 operations.

[9] The traditional pair-wise storage is also awkward for 4M algorithms during the packing of data from A and B, but this effect is not nearly as dramatic.

[10] This awkwardness will persist unless and until hardware architectures begin providing vector instructions for efficiently loading and storing non-contiguous elements in memory.

[11] A factor of two comes from the fact that, as shown in Eq. 2.2, 4M touches C^r and C^i twice each, while another factor of two comes from the cache blocksize scaling required on k_C in order to maintain the cache footprints of micro-panels of $\tilde{A}_i$ and $\tilde{B}_p$.

The algorithm captured by Figure 1.1 forms the basis for all level-3 implementations found in the BLIS framework (as of this writing). This algorithm is based on a so-called block-panel matrix multiplication.[12] The register (m_R, n_R) and cache (m_C, k_C, n_C) blocksizes labeled in the algorithmic diagram are typically chosen by the kernel developer as a function of hardware characteristics, such as the vector register set, cache sizes, and cache associativity. The authors of [16] present an analytical model for identifying suitable (if not optimal) values for these blocksizes.

3. 1m method. The primary motivation for seeking a better induced method comes from the observation that 4M inherently must update real and imaginary F.E. of C: (1) in separate steps, and may not use vector instructions to do so (due to the traditional pair-wise storage format); and (2) twice as frequently, in the case of 4M_1A, due to the algorithm's half-of-optimal cache blocksize k_C. As reviewed in Section 2.3, this imposes a significant drag on performance. If there existed an induced method that could update real and imaginary elements in one step, it may conveniently avoid both issues.

3.1. Derivation. Consider the classic definition of complex scalar multiplication and accumulation, shown in Eq. 2.1, refactored and expressed in terms of matrix and vector notation:

$$\begin{pmatrix} \gamma^r \\ \gamma^i \end{pmatrix} += \begin{pmatrix} \alpha^r & -\alpha^i \\ \alpha^i & \alpha^r \end{pmatrix} \begin{pmatrix} \beta^r \\ \beta^i \end{pmatrix} \tag{3.1}$$

Here, we have a singleton complex matrix multiplication problem that can naturally be expressed as a tiny real matrix multiplication where $m = k = 2$ and $n = 1$. Let us assume we implement this very small matrix multiplication according to the high-performance algorithm discussed in Section 2.4.

From this, we make the following key observation: If we pack α to $\tilde{A}_i$ in such a way that duplicates α^r and α^i to the second column of the micro-panel (while also swapping the placement of the duplicates and negating the duplicated α^i), and if we pack β to $\tilde{B}_p$ such that β^i is stored to the second row of the micro-panel (which, granted, only has one column), then a real domain GEMM micro-kernel executed on those micro-panels will compute the correct result in the complex domain and do so with a *single* invocation of that micro-kernel.

Thus, Eq. 3.1 serves as a packing template that hints at how the data must be stored. Furthermore, this template can be generalized. We augment α, β, γ with conventional row and column indices to denote the complex elements of matrices A, B, and C, respectively. Also, let us apply the Eq. 3.1 to the special case of $m = 3$, $n = 4$, and $k = 2$ to better observe the general pattern.

$$\begin{pmatrix} \gamma^r_{00} & \gamma^r_{01} & \gamma^r_{02} & \gamma^r_{03} \\ \gamma^i_{00} & \gamma^i_{01} & \gamma^i_{02} & \gamma^i_{03} \\ \gamma^r_{10} & \gamma^r_{11} & \gamma^r_{12} & \gamma^r_{13} \\ \gamma^i_{10} & \gamma^i_{11} & \gamma^i_{12} & \gamma^i_{13} \\ \gamma^r_{20} & \gamma^r_{21} & \gamma^r_{22} & \gamma^r_{23} \\ \gamma^i_{20} & \gamma^i_{21} & \gamma^i_{22} & \gamma^i_{23} \end{pmatrix} += \begin{pmatrix} \alpha^r_{00} & -\alpha^i_{00} & \alpha^r_{01} & -\alpha^i_{01} \\ \alpha^i_{00} & \alpha^r_{00} & \alpha^i_{01} & \alpha^r_{01} \\ \alpha^r_{10} & -\alpha^i_{10} & \alpha^r_{11} & -\alpha^i_{11} \\ \alpha^i_{10} & \alpha^r_{10} & \alpha^i_{11} & \alpha^r_{11} \\ \alpha^r_{20} & -\alpha^i_{20} & \alpha^r_{21} & -\alpha^i_{21} \\ \alpha^i_{20} & \alpha^r_{20} & \alpha^i_{21} & \alpha^r_{21} \end{pmatrix} \begin{pmatrix} \beta^r_{00} & \beta^r_{01} & \beta^r_{02} & \beta^r_{03} \\ \beta^i_{00} & \beta^i_{01} & \beta^i_{02} & \beta^i_{03} \\ \beta^r_{10} & \beta^r_{11} & \beta^r_{12} & \beta^r_{13} \\ \beta^i_{10} & \beta^i_{11} & \beta^i_{12} & \beta^i_{13} \end{pmatrix} \tag{3.2}$$

From this, we can make the following observations:

[12] This terminology describes the shape of the typical problem computed by the macro-kernel, i.e. the second loop around the micro-kernel. An alternative algorithm that casts its largest cache-bound subproblem in terms of panel-block matrix multiplication is discussed in [19].

- The complex matrix multiplication $C := C + AB$ with $m = 3$, $n = 4$, and $k = 2$ becomes a real matrix multiplication with $m = 6$, $n = 4$, and $k = 4$. In other words, the m and k dimensions are doubled for the purposes of the real GEMM primitive.
- If the primitive is the real GEMM micro-kernel, and we assume that matrices A and B above represent column-stored and row-stored micro-panels from $\tilde{A}_i$ and $\tilde{B}_p$, respectively, and also that the dimensions are conformal to the register blocksizes of this micro-kernel (i.e., $m = m_R$ and $n = n_R$) then the micro-panels of $\tilde{A}_i$ are packed from a $\frac{1}{2}m_R \times \frac{1}{2}k_C$ submatrix of A, which, when expanded in the special packing format, appears as the $m_R \times k_C$ micro-panel that the real GEMM micro-kernel expects.
- Similarly, the micro-panels of $\tilde{B}_p$ are packed from a $\frac{1}{2}k_C \times n_R$ submatrix of B, which, when reordered into a second special packing format, appears as the $k_C \times n_R$ micro-panel that the real GEMM micro-kernel expects.

It is easy to see by inspection that the real matrix multiplication implied by Eq. 3.2 induces the desired complex matrix multiplication. We will refer to the packing format used on matrix A above as the 1E format, since the F.E. are "expanded" (i.e., duplicated to the next column, with the duplicates swapped and the imaginary duplicate negated). Similarly, we will refer to the packing format used on matrix B above as the 1R format, since the F.E. are merely reordered (i.e., imaginary elements moved to the next row).

3.2. Two variants. Notice that implicit in the 1M method suggested by Eq. 3.2 is the fact that matrix C is stored by columns. This assumption is important; when A and B are packed according to the 1E and 1R formats, respectively, C must be stored by columns in order to allow the real domain primitive (or micro-kernel) to correctly update the individual real and imaginary F.E. of C with the corresponding F.E. from the matrix product AB.

Suppose that we instead refactored and expressed Eq. 2.1 as follows:

$$\begin{pmatrix} \gamma^r & \gamma^i \end{pmatrix} \mathrel{+}= \begin{pmatrix} \alpha^r & \alpha^i \end{pmatrix} \begin{pmatrix} \beta^r & \beta^i \\ -\beta^i & \beta^r \end{pmatrix} \tag{3.3}$$

This gives us a different template, one that implies different packing formats for matrices A and B. Applying Eq. 3.3 to the special case of $m = 4$, $n = 3$, and $k = 2$, yields:

(3.4)

$$\begin{pmatrix} \gamma^r_{00} & \gamma^i_{00} & \gamma^r_{01} & \gamma^i_{01} & \gamma^r_{02} & \gamma^i_{02} \\ \gamma^r_{10} & \gamma^i_{10} & \gamma^r_{11} & \gamma^i_{11} & \gamma^r_{12} & \gamma^i_{12} \\ \gamma^r_{20} & \gamma^i_{20} & \gamma^r_{21} & \gamma^i_{21} & \gamma^r_{22} & \gamma^i_{22} \\ \gamma^r_{30} & \gamma^i_{30} & \gamma^r_{31} & \gamma^i_{31} & \gamma^r_{32} & \gamma^i_{32} \end{pmatrix} \mathrel{+}= \begin{pmatrix} \alpha^r_{00} & \alpha^i_{00} & \alpha^r_{01} & \alpha^i_{01} \\ \alpha^r_{10} & \alpha^i_{10} & \alpha^r_{11} & \alpha^i_{11} \\ \alpha^r_{20} & \alpha^i_{20} & \alpha^r_{21} & \alpha^i_{21} \\ \alpha^r_{30} & \alpha^i_{30} & \alpha^r_{31} & \alpha^i_{31} \end{pmatrix} \begin{pmatrix} \beta^r_{00} & \beta^i_{00} & \beta^r_{01} & \beta^i_{01} & \beta^r_{02} & \beta^i_{02} \\ -\beta^i_{00} & \beta^r_{00} & -\beta^i_{01} & \beta^r_{01} & -\beta^i_{02} & \beta^r_{02} \\ \beta^r_{10} & \beta^i_{10} & \beta^r_{11} & \beta^i_{11} & \beta^r_{12} & \beta^i_{12} \\ -\beta^i_{10} & \beta^r_{10} & -\beta^i_{11} & \beta^r_{11} & -\beta^i_{12} & \beta^r_{12} \end{pmatrix}$$

In this variant, we see that matrix B, not A, is stored according to the 1E format (where columns become rows), while matrix A is stored according to 1R (where rows become columns). Also, we can see that matrix C must be stored by rows in order to allow the real GEMM micro-kernel to correctly update its F.E. with the corresponding values from the matrix product AB.

Henceforth, we will refer to the 1M variant exemplified in Eq. 3.2 as 1M_C since it is predicated on column storage of the output matrix C, and we will refer to the variant depicted in Eq. 3.4 as 1M_R since it assumes the output matrix is stored by rows.

TABLE 3.1
1M *complex domain blocksizes as a function of real domain blocksizes*

Variant	Blocksizes, in terms of real domain values, required for ...						
	k_C^z	m_C^z	n_C^z	m_R^z	m_P^z	n_R^z	n_P^z
1M_C	$\frac{1}{2}k_C$	$\frac{1}{2}m_C$	n_C	$\frac{1}{2}m_R$	m_P	n_R	n_P
1M_R	$\frac{1}{2}k_C$	m_C	$\frac{1}{2}n_C$	m_R	m_P	$\frac{1}{2}n_R$	n_P

Note: Blocksizes m_P and n_P represent the so-called "packing dimensions" for the micro-panels of $\tilde{A}_i$ and $\tilde{B}_p$, respectively. These values are analogous to the leading dimensions of matrices stored by columns or rows. In BLIS micro-kernels, typically $m_R = m_P$ and $n_R = n_P$, but sometimes the kernel author may find it useful for $m_R < m_P$ or $n_R < n_P$.

3.3. Determining complex blocksizes. As we alluded in Section 3.1, the appropriate blocksizes to use with 1M are a function of the real domain blocksizes. This makes sense, since the idea is to fool the real GEMM micro-kernel, and the various loops for register and cache blocking around the micro-kernel, into thinking that it is computing a real domain matrix multiplication. Which blocksizes must be modified (halved) and which are used unchanged depends on the variant of 1M being executed—or, more specifically, which matrix is packed according to the 1E format.

Table 3.1 summarizes the complex domain blocksizes prescribed for 1M_C and 1M_R as a function of the real domain values. This is somewhat analogous to the blocksize scaling described in Tables II and III in [21]. However, in that article the scaling was optional in the sense that different scaling factors would still work, albeit perhaps with a performance penalty. Here, in the case of Table 3.1, some scaling factors (namely, on m_R or n_R) are required in order for the 1M algorithm to function properly.

Those familiar with the matrix multiplication algorithm implemented by the BLIS framework, as depicted in Figure 1.1, may be unfamiliar with m_P and n_P, the so-called packing dimensions. These values are, effectively, the leading dimensions of the micro-panels. For most architectures, m_P and n_P are almost always equal to m_R and n_R, respectively. However, in some situations, it may be convenient (or necessary) to use $m_R < m_P$ or $n_R < n_P$. In any case, these packing dimensions are never scaled, even when their corresponding register blocksizes *are* scaled to accommodate the 1E format, because the halving that would otherwise be called for is cancelled out by the doubling of F.E. that manifests in the 1E format.

3.4. Algorithms.

3.4.1. General algorithm. Before investigating 1M method algorithms, we will first provide algorithms for computing real matrix multiplication to serve as a reference for the reader. Specifically, we provide pseudo-code, targeting the real domain, for the block-panel algorithm depicted in Figure 1.1. This algorithm is shown as RMMBP in Figure 3.1.

3.4.2. 1m-specific algorithm. When applied to the block-panel algorithm depicted in Figure 1.1, 1M_C and 1M_R yield nearly identical algorithms whose differences can be encoded within a few conditional statements within key parts of the high and low levels of code. We will refer to these specific algorithms as 1M_C_BP and 1M_R_BP,

Algorithm: $[\ C\ ] :=$ RMMBP(A, B, C)

for ($j = 0 : n-1 : n_C$)
  Identify B_j, C_j from B, C
  for ($p = 0 : k-1 : k_C$)
    Identify A_p, B_{jp} from A, B_j
    PACK $B_{jp} \rightarrow \tilde{B}_p$
    for ($i = 0 : m-1 : m_C$)
      Identify A_{pi}, C_{ji} from A_p, C_j
      PACK $A_{pi} \rightarrow \tilde{A}_i$
      for ($h = 0 : n_C-1 : n_R$)
        Identify $\tilde{B}_{ph}, C_{jih}$ from $\tilde{B}_p, C_{ji}$
        for ($l = 0 : m_C-1 : m_R$)
          Identify $\tilde{A}_{il}, C_{jihl}$ from $\tilde{A}_i, C_{jih}$
          $C_{jihl} :=$ RKERN($\tilde{A}_{il}$, $\tilde{B}_{ph}$, C_{jihl})

FIG. 3.1. *Abbreviated pseudo-code for implementing the general matrix multiplication algorithm depicted in Figure 1.1. Here,* RKERN *calls a real domain* GEMM *micro-kernel.*

Algorithm: $[\ C\ ] :=$ 1M_?_BP(A, B, C)

Set bool COLSTORE if RS(C) = 1
for ($j = 0 : n-1 : n_C$)
  Identify B_j, C_j from B, C
  for ($p = 0 : k-1 : k_C$)
    Identify A_p, B_{jp} from A, B_j
    if COLSTORE PACK1R $B_{jp} \rightarrow \tilde{B}_p$
    else PACK1E $B_{jp} \rightarrow \tilde{B}_p$
    for ($i = 0 : m-1 : m_C$)
      Identify A_{pi}, C_{ji} from A_p, C_j
      if COLSTORE PACK1E $A_{pi} \rightarrow \tilde{A}_i$
      else PACK1R $A_{pi} \rightarrow \tilde{A}_i$
      for ($h = 0 : n_C-1 : n_R$)
        Identify $\tilde{B}_{ph}, C_{jih}$ from $\tilde{B}_p, C_{ji}$
        for ($l = 0 : m_C-1 : m_R$)
          Identify $\tilde{A}_{il}, C_{jihl}$ from $\tilde{A}_i, C_{jih}$
          $C_{jihl} :=$ VK1M($\tilde{A}_{il}$, $\tilde{B}_{ph}$, C_{jihl})

$[\ C\ ] :=$ VK1M(A, B, C)

Acquire workspace W
Determine if using W; set USEW
if (USEW)
  Alias $C_{\text{use}} \leftarrow W$, $C_{\text{in}} \leftarrow 0$
else
  Alias $C_{\text{use}} \leftarrow C$, $C_{\text{in}} \leftarrow C$
Set bool COLSTORE if RS(C_{use}) = 1
if (COLSTORE) CS(C_{use}) $\times= 2$
else RS(C_{use}) $\times= 2$
N(A) $\times= 2$; M(B) $\times= 2$
$C_{\text{use}} :=$ RKERN(A, B, C_{in})
if (USEW)
  $C := W$

FIG. 3.2. *Left: Pseudo-code for Algorithms* 1M_C_BP *and* 1M_R_BP*, which result from applying* 1M_C *and* 1M_R *algorithmic variants to the block-panel algorithm depicted in Figure 1.1. Here,* PACK1E *and* PACK1R *pack matrices into the* 1E *and* 1R *formats, respectively. Right: Pseudo-code for a virtual micro-kernel used by all* 1M *algorithms.*

respectively. Figure 3.2 shows a hybrid algorithm that encompasses both, supporting row- and column-stored matrices C.

As with the 3M and 4M algorithms in [21], we have separated the so-called virtual micro-kernel into a separate function, shown in Figure 3.2 (right). This 1M-specific virtual micro-kernel, VK1M, largely consists of a call to the real domain micro-kernel

RKERN, with some added special case handling and book-keeping needed to properly induce complex matrix multiplication. Some of the details of the virtual micro-kernel will be addressed later.

3.5. Performance properties. Table 3.2 tallies the total number of F.E. memops required by the block-panel algorithm for both variants of 1M (1M_C and 1M_R). For comparison, we also include the corresponding memop counts for a selection of 4M algorithms as well as a conventional assembly-based solution, as first published in Table III in [21].

Notice that 1M_C_BP (and 1M_R_BP) incur additional memops relative to a conventional assembly-based solution. This stems from the fact that, unlike the assembly-based solution, 1M implementations cannot reuse[13] all real and imaginary F.E. from vector registers.

We can hypothesize that the observed performance signatures of 1M_C_BP and 1M_R_BP may be slightly different, because each places the additional memop overhead that is unique to 1M on different parts of the computation. This stems from the fact that there exists an asymmetry in the assignment of packing formats to matrices in each 1M variant. Specifically, 50% more memops—relative to a conventional assembly solution—are required during the initial packing and the movement between caches for the matrix packed according to 1E, since that format writes four F.E. for every two that it reads from the source operand. (Packing to 1R incurs the same number of memops as an assembly-based solution.) Also, if 1M_C_BP and 1M_R_BP use real micro-kernels with different micro-tile shapes (i.e., different values of m_R and n_R), those micro-kernels' differing performance properties will likely cause the performance signatures of 1M_C_BP and 1M_R_BP to deviate further.

Table 3.3 summarizes Table 3.2 and also lists the level of the memory hierarchy from which each matrix operand is reused as well as a measure of memory movement efficiency. The information listed for 4M and assembly algorithms is reproduced from Table IV of the previous article.

3.6. Algorithm details. This section lays out important details that must be handled when implementing the 1M method.

3.6.1. Micro-kernel I/O preference. Within the BLIS framework, micro-kernels are registered with a property that describes their input/output *preference*. The I/O preference describes whether the micro-kernel is set up to ideally use vector instructions to load and store elements of the micro-tile by rows or by columns. This property typically originates from the semantic orientation of vector registers used to accumulate the $m_R \times n_R$ micro-panel product. Whenever possible, the BLIS framework will perform logical transpositions[14] so that the apparent storage of C matches the preference property of the micro-kernel being used. This guarantees that the micro-kernel will be able to load and store F.E. of C using vector instructions.

This preference property is merely an interesting performance detail for conventional implementations (real and complex). However, in the case of 1M, it becomes important for constructing a correctly-functioning implementation. Specifically, the micro-kernel's I/O preference determines whether the 1M_C or 1M_R algorithm is prescribed. Generally speaking, a 1M_C algorithmic variant must employ a micro-kernel that prefers to access C by columns, while a 1M_R algorithmic variant must use a

[13] Here, the term "reuse" refers to the same reuse described in Footnote 7.

[14] This amounts to a swapping of the row and column strides and a swapping of the m and n dimensions.

TABLE 3.2
F.E. *memops incurred by various algorithms, broken down by stage of computation*

Algorithm[a]	F.E. memops required to ...[b]				
	update micro-tiles[c] C^r, C^i	pack $\tilde{A}_i$	move $\tilde{A}_i$ from L2 to L1 cache	pack $\tilde{B}_p$	move $\tilde{B}_p$ from L3 to L1 cache
4M_H	$8mn\frac{k}{k_C}$	$8mk\frac{n}{n_C}$	$4mk\frac{n}{n_R}$	$8kn$	$4kn\frac{m}{m_C}$
4M_1B	$8mn\frac{k}{k_C}$	$8mk\frac{2n}{n_C}$	$4mk\frac{n}{n_R}$	$8kn$	$4kn\frac{2m}{m_C}$
4M_1A	$8mn\frac{2k}{k_C}$	$8mk\frac{n}{n_C}$	$4mk\frac{n}{n_R}$	$8kn$	$4kn\frac{m}{m_C}$
assembly	$4mn\frac{k}{k_C}$	$4mk\frac{n}{n_C}$	$2mk\frac{n}{n_R}$	$4kn$	$2kn\frac{m}{m_C}$
1M_C_BP	$4mn\frac{2k}{k_C}$	$6mk\frac{n}{n_C}$	$4mk\frac{n}{n_R}$	$4kn$	$2kn\frac{2m}{m_C}$
1M_R_BP		$4mk\frac{n}{n_C}$	$2mk\frac{2n}{n_R}$	$6kn$	$4kn\frac{m}{m_C}$

[a] Algorithms 4M_H, 4M_1B, 4M_1A, and the assembly implementation employ a block-panel algorithm, and therefore their memop counts more closely resemble those of 1M_C_BP and 1M_R_BP.

[b] We express the number of iterations executed in the 5th, 4th, 3rd, and 2nd loops as $\frac{n}{n_C}$, $\frac{k}{k_C}$, $\frac{m}{m_C}$, and $\frac{n}{n_R}$. The precise number of iterations along a dimension x using a cache blocksize x_C would actually be $\lceil \frac{x}{x_C} \rceil$. Similarly, when blocksize scaling of $\frac{1}{2}$ is required, the precise value $\lceil \frac{x}{x_C/2} \rceil$ is expressed as $\frac{2x}{x_C}$. These simplifications allow easier comparison between algorithms while still providing meaningful approximations.

[c] As described in Section 3.6.2, $m_R \times n_R$ workspace sometimes becomes mandatory, such as when $\beta^i \neq 0$. When workspace is employed in a 4M-based algorithm, the number of F.E. memops incurred updating the micro-tile typically doubles from the values shown here.

micro-kernel that prefers to access C by rows.

3.6.2. Workspace. In some cases, a small amount of $m_R \times n_R$ workspace is needed. These cases fall into one of four scenarios: (1) C is row-stored and the real micro-kernel RKERN has a column preference; (2) C is column-stored and RKERN has a row preference; (3) C is general-stored (i.e., neither RS(C) nor CS(C) is unit); and (4) β has a non-zero imaginary component. If any of these situations apply, then the 1M virtual micro-kernel will need to use workspace. This corresponds to the setting of USEW in VK1M (in Figure 3.2). The idea is simply that RKERN will be called to compute the micro-panel product and store it to the workspace W. Subsequently, the result in W can be accumulated back to C.

Cases (1) and (2), while supported, actually never occur in practice because BLIS will perform (at a high level within the framework) a logical transposition whenever necessary. The net effect is that the storage of C will always appear to match the I/O preference of the micro-kernel.

Case (3) is needed because the real micro-kernel is programmed to support the updating of *real* matrices stored with general stride, which cannot be spoofed to match the updating of *complex* matrices stored with general stride. The reason is even when stored with general stride, complex matrices store real and imaginary F.E. in contiguous pairs. There is no way to coax this pattern of data access from a real domain micro-kernel, given its existing API. Thus, general stride support must be implemented outside RKERN, within VK1M.

Case (4) is needed because real domain micro-kernels are not capable of scaling

TABLE 3.3
Performance properties of various algorithms

Algorithm	Total F.E. memops required (Sum of columns of Table 3.2)	Level from which F.E. of matrix X are reused, and l_{L1}: # of times each cache line is moved into the L1 cache (per rank-k_C update).					
		C	l_{L1}^C	A	l_{L1}^A	B	l_{L1}^B
4M_H	$8mn\left(\frac{k}{k_C}\right)+4mk\left(\frac{2n}{n_C}+\frac{n}{n_R}\right)+2kn\left(4+\frac{2m}{m_C}\right)$	MEM	4	MEM	4	MEM	4
4M_1B	$8mn\left(\frac{k}{k_C}\right)+4mk\left(\frac{4n}{n_C}+\frac{n}{n_R}\right)+2kn\left(4+\frac{4m}{m_C}\right)$	L2	2[a]	L2	1	L1	1
4M_1A	$8mn\left(\frac{2k}{k_C}\right)+4mk\left(\frac{2n}{n_C}+\frac{n}{n_R}\right)+2kn\left(4+\frac{2m}{m_C}\right)$	L1	1[a]	L1	1	L1	1
assembly	$4mn\left(\frac{k}{k_C}\right)+2mk\left(\frac{2n}{n_C}+\frac{n}{n_R}\right)+2kn\left(2+\frac{m}{m_C}\right)$	REG	1	REG	1	REG	1
1M_C_BP	$4mn\left(\frac{2k}{k_C}\right)+2mk\left(\frac{3n}{n_C}+\frac{2n}{n_R}\right)+2kn\left(2+\frac{2m}{m_C}\right)$	REG	1	L2[b]	1	REG	1
1M_R_BP	$4mn\left(\frac{2k}{k_C}\right)+2mk\left(\frac{2n}{n_C}+\frac{2n}{n_R}\right)+2kn\left(3+\frac{2m}{m_C}\right)$	REG	1	REG	1	L1[b]	1

[a] This assumes that the micro-tile is not evicted from the L1 cache during the next call to RKERN.

[b] In the case of 1M algorithms, we consider F.E. of A and B to be "reused" from the level of cache in which the 1E-formatted matrix resides.

C by complex scalars β (that is, β such that $\beta^i \neq 0$).

3.6.3. Handling alpha and beta scalars. As in the previous article, we have simplified the general matrix multiplication to $C := C+AB$. In practice, the operation is implemented as $C := \beta C + \alpha AB$, where $\alpha, \beta \in \mathbb{C}$. Let us use Algorithms 1M_C_BP and 1M_R_BP in Figure 3.2 as an example of how to support arbitrary values of α and β.

If no workspace is needed (because none of the four situations described in Section 3.6.2 apply), we can simply pass β^r into the RKERN call. However, if β is complex, or (regardless of whether β is real or complex) if any of the other three workspace cases apply, then we must pass in a local $\beta_{\text{use}} = 0$ to RKERN, compute to local workspace W, and then apply β at the end of VK1M, when W is accumulated to C.

When α is real, the scaling may be performed directly by RKERN. This situation is ideal since it almost always incurs no additional costs (since many micro-kernels multiply their intermediate AB product by α unconditionally). Scaling by α with nonzero imaginary components can be performed by the packing function when either $\tilde{A}_i$ or $\tilde{B}_p$ are packed. Though somewhat less than ideal, the overhead incurred by this treatment of α is minimal since packing is a memory-bound operation.

3.6.4. Multithreading. As with Algorithm 4M_1A in the previous article, Algorithms 1M_C_BP and 1M_R_BP parallelize in a straightforward manner for multicore and many-core environments. Because those algorithms encode the 1M method entirely within the packing functions and the virtual micro-kernel, all other levels of code are completely oblivious to, and therefore unaffected by, the specifics of the new algorithms. Therefore, we expect that 1M_C_BP and 1M_R_BP will yield multithreaded performance that is on-par with that of the corresponding real domain matrix multiplication function, RMMBP.

3.6.5. Bypassing the virtual micro-kernel. Because the 1M virtual micro-kernel serves as a wrapper to the real domain micro-kernel, it would seem at first glance that there exists the potential for additional overhead in 1M algorithms, particularly from the extra function calls. However, there are a few things to consider.

First, we should consider that a conventional solution would implement matrix multiplication using a complex micro-kernel, which usually has a smaller micro-tile footprint (i.e., fewer F.E.). But, a complex micro-kernel that updates fewer F.E. would need to be called more times per rank-k_C update in order to fully update the output matrix C. Thus, the function call overhead incurred by 1M algorithms may already be at or near parity with that of a conventional implementation.

Secondly, even if the complex micro-kernel updates the same number of F.E. as its real domain counterpart, there exists a simple optimization that can be applied as long as $\beta^i = 0$ and C is either row- or column-stored (but not general-stored). If these conditions are met and detected by the implementation, the real domain *macro*-kernel can be called with modified parameters to induce the equivalent complex domain subproblem. This simple optimization avoids all overhead introduced by the virtual micro-kernel, including (but not limited to) the cost incurred by additional function calls.

Finally, we suspect that, even if this optimization cannot be applied, the slowdown that results from the additional overhead may be tolerable to many applications.

3.7. 2m. The previous article noted that because of its expression in terms of real and imaginary matrices, 4M would perform more favorably on complex matrices that stored real and imaginary values separately, as two real matrices. This would not benefit the accesses on A and B since those matrices must almost always be packed to achieve high performance and can easily be separated during that process. However, it would benefit the accesses on C. Referring back to Eq. 3.2, we can see that storing the real and imaginary F.E. of C separately is equivalent to a permutation of the rows of C. In order to keep the computation expressed unchanged, this permutation would need to be applied to matrix A as well since they both share an m dimension. Applying such a permutation to Eq. 3.2 yields:

$$\left(\begin{array}{cccc} \gamma_{00}^r & \gamma_{01}^r & \gamma_{02}^r & \gamma_{03}^r \\ \gamma_{10}^r & \gamma_{11}^r & \gamma_{12}^r & \gamma_{13}^r \\ \gamma_{20}^r & \gamma_{21}^r & \gamma_{22}^r & \gamma_{23}^r \\ \hline \gamma_{00}^i & \gamma_{01}^i & \gamma_{02}^i & \gamma_{03}^i \\ \gamma_{10}^i & \gamma_{11}^i & \gamma_{12}^i & \gamma_{13}^i \\ \gamma_{20}^i & \gamma_{21}^i & \gamma_{22}^i & \gamma_{23}^i \end{array}\right) \mathrel{+}= \left(\begin{array}{cccc} \alpha_{00}^r & -\alpha_{00}^i & \alpha_{01}^r & -\alpha_{01}^i \\ \alpha_{10}^r & -\alpha_{10}^i & \alpha_{11}^r & -\alpha_{11}^i \\ \alpha_{20}^r & -\alpha_{20}^i & \alpha_{21}^r & -\alpha_{21}^i \\ \hline \alpha_{00}^i & \alpha_{00}^r & \alpha_{01}^i & \alpha_{01}^r \\ \alpha_{10}^i & \alpha_{10}^r & \alpha_{11}^i & \alpha_{11}^r \\ \alpha_{20}^i & \alpha_{20}^r & \alpha_{21}^i & \alpha_{21}^r \end{array}\right) \left(\begin{array}{cccc} \beta_{00}^r & \beta_{01}^r & \beta_{02}^r & \beta_{03}^r \\ \beta_{00}^i & \beta_{01}^i & \beta_{02}^i & \beta_{03}^i \\ \beta_{10}^r & \beta_{11}^r & \beta_{12}^r & \beta_{13}^r \\ \beta_{10}^i & \beta_{11}^i & \beta_{12}^i & \beta_{13}^i \end{array}\right) \tag{3.5}$$

If we also permute even-indexed columns of A to be consecutive with one another (and grouped together) while odd-numbered columns are permuted to immediately follow them, and also permute rows of B accordingly, we have:

$$\left(\begin{array}{cccc} \gamma_{00}^r & \gamma_{01}^r & \gamma_{02}^r & \gamma_{03}^r \\ \gamma_{10}^r & \gamma_{11}^r & \gamma_{12}^r & \gamma_{13}^r \\ \gamma_{20}^r & \gamma_{21}^r & \gamma_{22}^r & \gamma_{23}^r \\ \hline \gamma_{00}^i & \gamma_{01}^i & \gamma_{02}^i & \gamma_{03}^i \\ \gamma_{10}^i & \gamma_{11}^i & \gamma_{12}^i & \gamma_{13}^i \\ \gamma_{20}^i & \gamma_{21}^i & \gamma_{22}^i & \gamma_{23}^i \end{array}\right) \mathrel{+}= \left(\begin{array}{cc|cc} \alpha_{00}^r & \alpha_{01}^r & -\alpha_{00}^i & -\alpha_{01}^i \\ \alpha_{10}^r & \alpha_{11}^r & -\alpha_{10}^i & -\alpha_{11}^i \\ \alpha_{20}^r & \alpha_{21}^r & -\alpha_{20}^i & -\alpha_{21}^i \\ \hline \alpha_{00}^i & \alpha_{01}^i & \alpha_{00}^r & \alpha_{01}^r \\ \alpha_{10}^i & \alpha_{11}^i & \alpha_{10}^r & \alpha_{11}^r \\ \alpha_{20}^i & \alpha_{21}^i & \alpha_{20}^r & \alpha_{21}^r \end{array}\right) \left(\begin{array}{cccc} \beta_{00}^r & \beta_{01}^r & \beta_{02}^r & \beta_{03}^r \\ \beta_{10}^r & \beta_{11}^r & \beta_{12}^r & \beta_{13}^r \\ \hline \beta_{00}^i & \beta_{01}^i & \beta_{02}^i & \beta_{03}^i \\ \beta_{10}^i & \beta_{11}^i & \beta_{12}^i & \beta_{13}^i \end{array}\right)$$

Or, more generally:

$$\left(\begin{array}{c} C^r \\ \hline C^i \end{array}\right) \mathrel{+}= \left(\begin{array}{c|c} A^r & -A^i \\ \hline A^i & A^r \end{array}\right) \left(\begin{array}{c} B^r \\ \hline B^i \end{array}\right) \tag{3.6}$$

This matrix multiplication can be computed via just *two* calls to a real domain matrix multiplication primitive:

$$C^r \mathrel{+}= \left(\begin{array}{c|c} A^r & -A^i \end{array}\right) \left(\begin{array}{c} B^r \\ \hline B^i \end{array}\right), \quad C^i \mathrel{+}= \left(\begin{array}{c|c} A^i & A^r \end{array}\right) \left(\begin{array}{c} B^r \\ \hline B^i \end{array}\right)$$

provided that $\left(\begin{array}{c|c} A^r & -A^i \end{array}\right)$, $\left(\begin{array}{c|c} A^i & A^r \end{array}\right)$, and $\left(\begin{array}{c} B^r \\ \hline B^i \end{array}\right)$ can each be stored so that they can be referenced as single matrices. We call this the 2M method and refer to the specific instance derived from Eq. 3.6 as 2M_C.

Applying similar permutations to the n and k dimensions to Eq. 3.4 yields:

$$\left(\begin{array}{c|c} C^r & C^i \end{array}\right) \mathrel{+}= \left(\begin{array}{c|c} A^r & A^i \end{array}\right) \left(\begin{array}{c|c} B^r & B^i \\ \hline -B^i & B^r \end{array}\right) \tag{3.7}$$

which can also be broken down in two instances of real matrix multiplication:

$$C^r \mathrel{+}= \left(\begin{array}{c|c} A^r & A^i \end{array}\right) \left(\begin{array}{c} B^r \\ \hline -B^i \end{array}\right), \quad C^i \mathrel{+}= \left(\begin{array}{c|c} A^r & A^i \end{array}\right) \left(\begin{array}{c} B^i \\ \hline B^r \end{array}\right)$$

which corresponds to 2M_R.

We can now make a few observations about 2M:

- Eqs. 3.6 and 3.7 are identical to Eqs. 3.1 and 3.3, respectively, except that scalars are replaced with matrices.
- The permutation along the k dimension is actually unnecessary and so the formatting captured by Eq. 3.5 also falls under 2M. This permutation (or lack thereof) only changes the order in which intermediate terms are accumulated.
- The storage of C^r and C^i in both Eqs. 3.6 and 3.7 is unspecified and does not depend on which matrix—A or B—was originally formatted with 1E (prior to permutation). Either C^r or C^i may be stored by rows, columns, or a more exotic storage scheme, and their storage formats need not even be identical. Thus, neither 2M_C nor 2M_R implies the storage of C; rather, they only imply how matrices A and B are formatted and stored—that is, which one contains duplicated (and negated) F.E..
- The 2M method can be applied at an arbitrary level of matrix multiplication. For example, if we assume from Eq. 3.6 that input matrices $\left(\begin{array}{c|c} A^r & -A^i \end{array}\right)$, $\left(\begin{array}{c|c} A^i & A^r \end{array}\right)$, and $\left(\begin{array}{c} B^r \\ \hline B^i \end{array}\right)$ are each stored as micro-panels (stored by columns, columns, and rows, respectively), then the implied primitive is the real GEMM micro-kernel, which will update $m_R \times n_R$ micro-tiles of C.

4. Performance. In this section we present performance results for implementations of 1M algorithms on a recent Intel architecture. For comparison, we include results for a 4M algorithm as well as conventional assembly-based approaches in the real and complex domains.

TABLE 4.1
Register and cache blocksizes used by various BLIS implementations of matrix multiplication, as configured for an Intel Xeon E5-2690 v3 "Haswell" processor

Precision/Domain	Implementation	m_R^z	n_R^z	m_C^z	k_C^z	n_C^z
single complex	BLIS 1M_C	16/2	6	144/2	256/2	4080
	BLIS 1M_R	6	16/2	144	256/2	4080/2
	BLIS assembly (c)	8	3	56	256	4080
	BLIS assembly (r)	3	8	75	256	4080
double complex	BLIS 1M_C	8/2	6	72/2	256/2	4080
	BLIS 1M_R	6	8/2	72	256/2	4080/2
	BLIS assembly (c)	4	3	44	256	4080
	BLIS assembly (r)	3	4	192	256	4080

Note: For 1M implementations, division by 2 is made explicit to allow the reader to quickly see both the complex blocksize values as well as the values that would be used by the underlying real domain micro-kernels when performing real matrix multiplication. The I/O preference of the assembly-based implementations is indicated by a "(c)" or "(r)" (for column- or row-preferring).

4.1. Platform and implementation details. Results presented in this section were gathered on a single Cray XC40 compute node consisting of two 12-core Intel Xeon E5-2690 v3 processors featuring the "Haswell" microarchitecture. Each core, running at a clock rate of 3.2 GHz[15], provides a single-core peak performance of 51.2 gigaflops (GFLOPS) in double precision and 102.4 GFLOPS in single precision.[16] Each socket has a 30MB L3 cache that is shared among cores, and each core has a private 256KB L2 cache and 32KB L1 (data) cache. Performance experiments were gathered under the Cray Linux Environment 6 operating system running the Linux 4.4.103 (x86_64) kernel. Source code was compiled by the GNU C compiler (`gcc`) version 7.3.0.[17] The version of BLIS used in these tests was not officially released at the time of this writing, and was adapted from version 0.6.0-11.[18]

Algorithms 1M_C_BP and 1M_R_BP were implemented in the BLIS framework, as described in Section 3.4. We also refer to results based on existing conventional assembly-based micro-kernels written by hand for the Haswell microarchitecture via GNU extended inline assembly syntax.

All experiments were performed on randomized, column-stored matrices with GEMM scalars held constant: $\alpha = \beta = 1$. In all performance graphs, each data point represents the best of three trials.

Blocksizes for each of the BLIS implementations tested are provided in Table 4.1.

In all graphs presented in this section the x-axes denote the problem size, the y-axes show observed floating-point performance in units of GFLOPS per core, and

[15] This system uses Intel's Turbo Boost 2.0 dynamic frequency throttling technology. According to [15], the maximum the clock frequency when executing AVX instructions is 3.2 GHz when utilizing one or two cores, and 3.0 GHz when utilizing three or more cores.

[16] Accounting for the reduced AVX clock frequency, the peak performance when utilizing 24 cores is 48 GFLOPS/core in double precision and 96 GFLOPS/core in single precision.

[17] The following optimization flags were used during compilation of BLIS and its test drivers: `-O3 -mavx2 -mfma -mfpmath=sse -march=haswell`.

[18] Despite not yet having an official version number, this version of BLIS may be uniquely identified, with high probability, by the first 10 digits of its `git` "commit" (SHA1 hash) number: `ceee2f973e`.

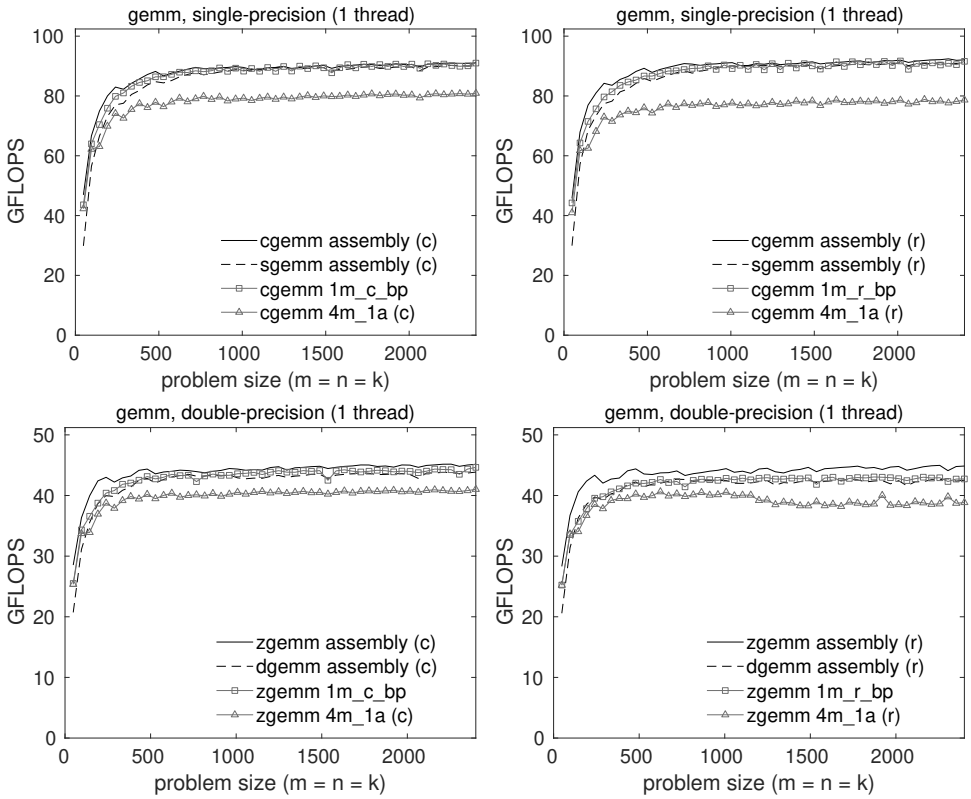

FIG. 4.1. *Single-threaded performance of various implementations of single-precision (top) and double-precision (bottom) complex* GEMM *on a single core of an Intel Xeon E5-2690 v3 "Haswell" processor. The left and right graphs differ in which* 1M *implementation they report, with the left graphs reporting* 1M_C_BP *(which employs a column-preferring micro-kernel) and the right graphs reporting* 1M_R_BP *(which employs a row-preferring micro-kernel). The graphs also contain three reference curves for comparison: an assembly-coded complex* GEMM*, an assembly-coded real* GEMM*, and the* 4M_1A *implementation found in BLIS (with the latter two using the same micro-kernel as the* 1M *implementation shown in the same graph). For consistency with the* 1M *curves, these reference implementations differ from left to right graphs in the I/O preference of their underlying micro-kernel, indicated by a "(c)" or "(r)" (for column- or row-preferring) in the legends. The theoretical peak performance coincides with the top of each graph.*

the theoretical peak performance coincides with the top of each graph.

4.2. Sequential results. Figure 4.1 reports performance results for various implementations of double- and single-precision complex matrix multiplication on a single core of the Haswell processor. For these results, all matrix dimensions were equal (e.g. $m = n = k$). Results for 1M_C_BP (which uses a column-preferring micro-kernel) appears on the left of Figure 4.1 while those of 1M_R_BP (which uses a row-preferring micro-kernel) appears on the right.

Each graph in Figure 4.1 also contains three reference implementations: BLIS's complex GEMM based on conventional assembly-coded kernels (e.g. "cgemm assembly"); BLIS's real GEMM (e.g. "sgemm assembly"); and the 4M_1A implementation found in BLIS.[19] We configured all three of these reference codes to use column-

[19] Within any given graph of Figures 4.1 and 4.2, the 1M and 4M_1A implementations use the same

preferential micro-kernels on the left and row-preferential micro-kernels on the right, as indicated by a "(c)" or "(r)" in the legends, in order to provide consistency with the 1M results.

As predicted in Section 3.5, we find that the performance signatures of the 1M_C_BP and 1M_R_BP algorithms differ slightly. This was expected given that the 1E and 1R packing formats place different memory access burdens on different packed matrices, $\tilde{A}_i$ and $\tilde{B}_p$, which reside in different levels of cache. It was not previously clear, however, which would be superior over the other. It seems that, at least in the sequential case, the difference is somewhat more noticeable in double-precision, though even there it is quite subtle. This difference is almost certainly due to the individual performance characteristics of the underlying row- and column-preferential micro-kernels. We find evidence of this in the 4M_1A results, which was also affected by the change in micro-kernel I/O preference.

In all cases, the 1M implementations outperform 4M_1A, with the margin somewhat larger in single-precision.

The 1M implementations match or exceed the performance of their real domain GEMM benchmarks (the dotted lines in each graph), and are quite competitive with assembly-coded complex GEMM (the solid lines) regardless of the algorithm employed.

4.3. Multithreaded results. Figure 4.2 shows single- and double-precision performance using 24 threads, with one thread bound to each physical core of the processor. Performance is presented in units of gigaflops per core to facilitate visual assessment of scalability. For all BLIS implementations, we employed 4-way parallelism within the 5th loop, 3-way parallelism within the 3rd loop, and 2-way parallelism in the 2nd loop for a total of 24 threads. This parallelization scheme was chosen in a manner consistent with that of the previous article using a strategy set forth in [18].

Compared to the single-threaded case, we find a more noticable difference in multithreaded performance between the 1M algorithms. Specifically, the 1M_R_BP implementation (based on a row-preferring micro-kernel) outperforms that of 1M_C_BP (based on a column-preferring micro-kernel), with the difference more pronounced in single-precision. We suspect this is rooted not in the algorithms, *per se*, but in the differing micro-kernel implementations used by each 1M algorithm. The 1M_R_BP algorithm is implemented with a real micro-kernel that is 6×16 and 6×8 in the single- and double-precision cases, respectively, while 1M_C_BP uses 16×6 and 8×6 micro-kernels for single- and double-precision implementations, respectively. The observed difference in performance between the 1M algorithms is likely attributable to the fact that the micro-kernels' different values for m_R and n_R place different bandwidth requirements when reading F.E. from the caches (primarily L1 and L2). More specifically, larger values of m_R place a heavier burden on loading elements from the L2 cache, which is usually disadvantageous since that cache can typically sustain lower bandwidth. By contrast, a micro-kernel with larger n_R loads more elements (per $m_R \times n_R$ rank-1 update) from the L1 cache, which resides closer to the processor and can typically sustain higher bandwidth than the L2 cache.

The multithreaded 1M implementation approximately matches or exceeds its real domain counterpart in all cases.

The 1M algorithm based on a row-preferential micro-kernel, 1M_R_BP, outperforms 4M_1A, especially in single-precision where the margin is quite wide. The 1M algorithm based on column-preferential micro-kernels, 1M_C_BP, performs more

real-domain micro-kernel that of the real GEMM (e.g. "sgemm assembly" or "dgemm assembly").

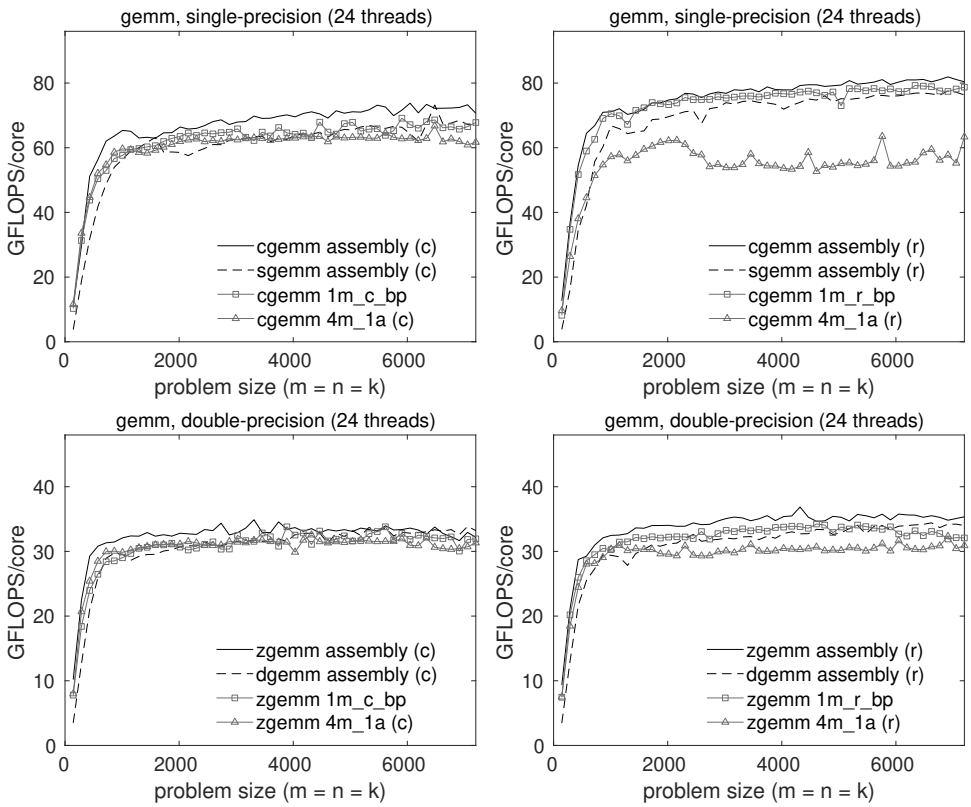

FIG. 4.2. *Multithreaded performance of various implementations of single-precision (top) and double-precision (bottom) complex* GEMM *on two Intel Xeon E5-2690 v3 "Haswell" processors, each with 12 cores. All data points reflect the use of 24 threads. The left and right graphs differ in which* 1M *implementation they report, with the left graphs reporting* 1M_C_BP *(which employs a column-preferring micro-kernel) and the right graphs reporting* 1M_R_BP *(which employs a row-preferring micro-kernel). The graphs also contain three reference curves for comparison: an assembly-coded complex* GEMM*, an assembly-coded real* GEMM*, and the* 4M_1A *implementation found in BLIS (with the latter two using the same micro-kernel as the* 1M *implementation shown in the same graph). For consistency with the* 1M *curves, these reference implementations differ from left to right graphs in the I/O preference of their underlying micro-kernel, indicated by a "(c)" or "(r)" (for column- or row-preferring) in the legends. The theoretical peak performance coincides with the top of each graph.*

poorly, barely ediging out 4M_1A in single precision and tracking closely with 4M_1A in double precision. We suspect that 4M_1A is more resilient to the lower-performing column-preferential micro-kernel by virtue of the fact that the algorithm's virtual micro-kernel leans heavily on the L1 cache, which on this architecture is capable of being read from and written to at relatively high bandwidth (64 bytes/cycle and 32 bytes/cycle, respectively) [14].

4.4. Comparing to other implementations. While our primary goal is not to compare the performance of the newly developed 1M implementations with that of other established BLAS solutions, some basic comparison is merited and thus we have included Figure 4.3 (left). These graphs are similar to those in Figure 4.1, except that: we show only implementations based on row-preferential micro-kernels; we omit 4M_1A; and we include results for complex GEMM implementations provided

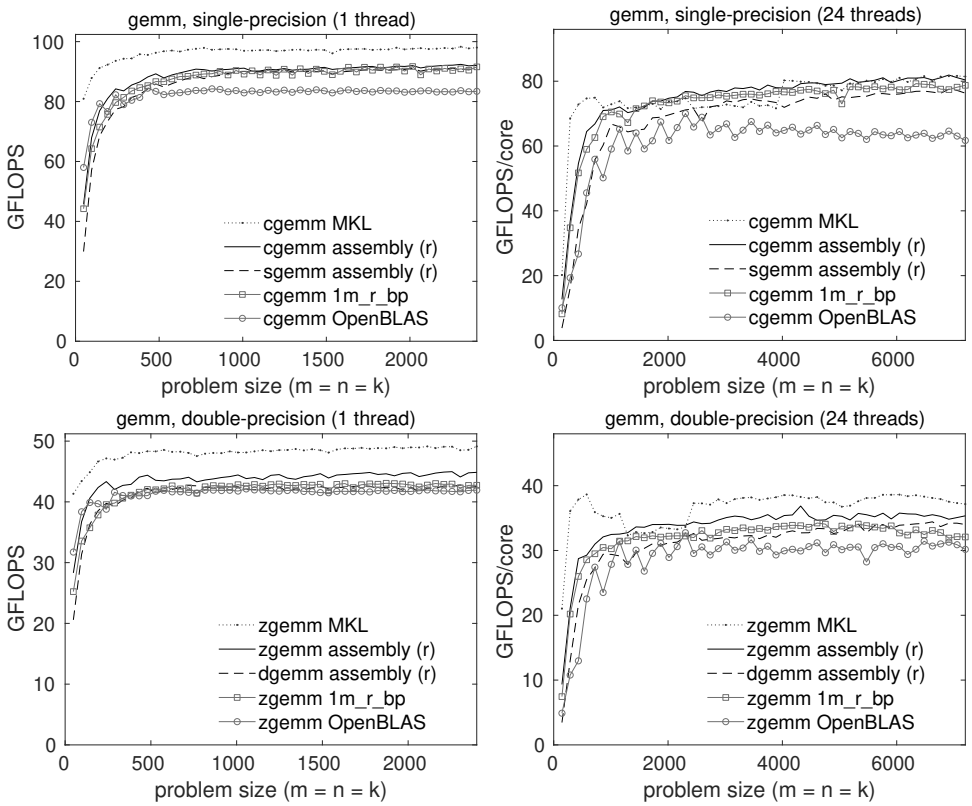

FIG. 4.3. *Single-threaded (left) and multithreaded (right) performance of various implementations of single-precision (top) and double-precision (bottom) complex* GEMM *on a single core (left) or 12 cores (right) of an Intel Xeon E5-2690 v3 "Haswell" processor. All multithreaded data points reflect the use of 24 threads. The* 1M *curves are identical from those shown in Figures 4.1 and 4.2. The theoretical peak performance coincides with the top of each graph.*

by OpenBLAS 0.3.6 [17] and Intel MKL 2019 Update 4 [13].

Figure 4.3 (right) shows multithreaded performance of the same implementations running with 24 threads.

These graphs show that BLIS's complex assembly-based and 1M implementations typically outperform OpenBLAS while falling short in most (but not all) cases when compared to Intel's MKL library.

4.5. Additional results. Additional performance results were gathered on a 56-core Marvell ThunderX2 compute server. For brevity, we present and discuss that data in Appendix A. Those results reinforce the narrative provided here, lending even more evidence that the 1M method is capable of yielding high-performance implementations of complex matrix multiplication that are competitive with (and often outperform) other leading library solutions.

5. Observations.

5.1. 4m limitations circumvented. The previous article concluded by identifying a number of limitations inherent in the 4M method that collectively prevent the approach from becoming both a feasible and competitive alternative to matrix

multiplication via conventional assembly-based kernels. We now revisit this list and briefly discuss whether, to what degree, and how those limitations are overcome by algorithms based on the 1M method.

Number of calls to primitive. The most versatile 4M algorithm, 4M_1A, incurs up to a four-fold increase in function call overhead over a comparable assembly-based implementation. By comparison, 1M algorithms require at most a doubling of micro-kernel function call overhead, and in certain common cases (e.g., when $\beta \in \mathbb{R}$ and C is row- or column-stored), this overhead can be avoided completely. The 1M method is clearly an improvement over 4M due to its reliance on a single invocation of the matrix multiplication primitive.

Inefficient reuse of input data from A, B, and C. The most cache-efficient application of 4M is the lowest level algorithm, 4M_1A, which reuses F.E. of A, B, and C from the L1 cache. But, as shown in Table 3.3, both 1M_R and 1M_C variants reuse F.E. of two of the three matrices from registers, with 1M_R_BP reusing F.E. of the third matrix from the L1 cache. This would seem to be a significant improvement, though observed performance improvement over 4M_1A will depend on properties specific to the hardware.

Non-contiguous output to C. Algorithms based on the 4M method must update only the real and then only the imaginary parts of the output matrix, twice each. Since C is typically stored (by rows or columns) with real and imaginary F.E. interleaved, this piecemeal approach prevents the real micro-kernel from using vector load and store instructions on C during those four updates. The 1M method avoids this issue altogether by packing A and B to formats that allow the real micro-kernel to update contiguous real and imaginary F.E. of C simultaneously within a single invocation. We suspect that this is, perhaps, the largest contributor to 1M's performance superiority over 4M.

Reduction of k_C. Algorithm 4M_1A requires that the real micro-kernel's preferred k_C blocksize be halved in the complex algorithm in order to maintain proper cache footprints of $\tilde{A}_i$ and $\tilde{B}_p$ as well the footprints of their constituent micro-panels.[20] Using such sub-optimally sized micro-panels can noticeably hobble the performance of 4M_1A. Looking back at Table 3.1, it may seem like 1M suffers a similar handicap; however, the reason for halving k_C and its effect are both completely different. In the case of 1M, the use of $k_C^z = \frac{1}{2}k_C$ is simply a conversion of units (complex elements to real F.E.) for the purposes of identifying the size of the complex submatrices to be packed that will induce the optimal k_C value from the perspective of the real micro-kernel, *not* a reduction in the F.E. footprint of the micro-panels operated upon by that real micro-kernel. Indeed, the ability of 1M to achieve high performance when $k = \frac{1}{2}k_C$ is a strength in the context of certain higher-level applications, such as Cholesky, LU, and QR factorizations based on rank-k update. Those operations tend to perform better when the algorithmic blocksize (corresponding to k_C) is as narrow as possible in order to limit the amount of computation in the lower-performing unblocked subproblem.

Framework accommodation. The 1M algorithms are no more disruptive to the BLIS framework than the most accommodating of 4M algorithms, 4M_1A, and much

[20] Recall that the halving of k_C for 4M_1A was motivated by the desire to keep not just two, but four real micro-panels in the L1 cache simultaneously. These correspond to the real and imaginary parts of the current micro-panels of $\tilde{A}_i$ and $\tilde{B}_p$.

less disruptive than the remaining algorithms. This is because, like with 4M_1A, almost all of the 1M implementation details are sequestered within the packing routines and the virtual micro-kernel.

Interference with multithreading. Because the 1M algorithms are implemented entirely within the packing facility and virtual micro-kernel, they parallelize just as easily as the most thread-friendly of the 4M algorithms, 4M_1A, and entirely avoid the threading difficulties of higher-level 4M algorithms.[21]

Non-applicability to two-operand operations. Certain higher-level applications of 4M are inherently incompatible with two-operand operations because they would overwrite the original contents of the input/output operand even though subsequent stages of computation depend on that original input. 1M avoids this limitation entirely. Like 4M_1A, 1M can easily be applied to two-operand level-3 operations such as TRMM and TRSM.[22]

Summary. The analysis above suggests that the 1M method solves or avoids most of the performance-degrading weaknesses of 4M and in the remaining cases is no worse off than the best 4M algorithm. Thus, its observed performance superiority was predictable.

5.2. Further discussion. Before concluding, here we offer some final thoughts on the 1M method and its place in the larger spectrum of approaches to implementing complex matrix multiplication.

5.2.1. Geometric interpretation. Matrix multiplication is sometimes thought of as a three-dimensional operation with a contraction (accumulation) over the k dimension. This interpretation carries into the complex domain as well. However, when each complex element is viewed in terms of its real and imaginary components, we find that a fourth pseudo-dimension of computation (of fixed size 2) emerges, one which also involves a contraction. The 1M method reorders and duplicates elements of A and B in such a way that exposes and "flattens" this extra dimension of computation. This, combined with the exposed treatment of real and imaginary F.E., causes the resulting floating-point operations to appear indistinguishable from a real domain matrix multiplication with m and k dimensions (for column-stored C) or k and n dimensions (for row-stored C) that are twice as long.

5.2.2. Data reuse: efficiency vs. programmability. Both the conventional approach and 1M move data efficiently through the memory hierarchy.[23] However, once in registers, a conventional complex micro-kernel reuses those loaded values to perform twice as many flops as 1M. The previous article observes that all 4M algorithms make different variations of the same tradeoff: by forgoing the reuse of F.E. from registers and instead reusing those data from some level of cache, the algorithms avoid the need to explicitly encode complex arithmetic at the assembly level. As it turns out, 1M makes a similar tradeoff, but gives up less while gaining more: it is able to effectively reuse F.E. from two of the three matrix operands from registers while still avoiding the need for a complex micro-kernel, and it manages

[21] This thread-friendly property holds even when the virtual micro-kernel is bypassed altogether as discussed in Section 3.6.5

[22] As with 4M_1A, 1M support for TRSM requires a separate pair of virtual micro-kernels that fuse a matrix multiplication with a triangular solve with n_R right-hand sides.

[23] This is in contrast to, for example, Algorithm 4M_HW, which the previous article showed makes rather inefficient use of cache lines as they travel through the L3, L2, and L1 caches.

to replace that kernel operation with a single real matrix multiplication. And we would argue that increasing programmability and productivity by forfeiting a modest performance advantage is a good trade to make under almost any circumstance.

5.2.3. Micro-kernel bandwidth. Because 1M algorithms do not *explicitly* reuse F.E. of $\tilde{A}_i$ and $\tilde{B}_p$ from registers, they require higher memory bandwidth during the micro-kernel computation than a conventional assembly-based solution.[24] Specifically, increased bandwidth is utilized when reading from the copy of the matrix that is packed into the 1E format, which resides in either the L2 or L1 cache, depending on which algorithm is being employed.[25] In practice, this potential weakness of 1M is not a concern. Yes, in principle, one could design an architecture with sufficiently low memory bandwidth from L2 or L1 cache that a conventional complex matrix multiplication achieves high performance while a 1M-based implementation struggles. However, this would imply a corresponding performance shortfall in the underlying real domain matrix kernel. Given the ubiquity and importance of real matrix multiplication in the scientific community, hardware vendors have great incentive to design architectures that allow `sgemm` and `dgemm` to achieve high performance. Thus, we would expect that 1M will remain a viable alternative for the foreseeable future.

5.2.4. Storage. The supremacy of the 1M method is closely tied to the interleaved, pairwise storage of real and imaginary values—specifically, of the output matrix C. If users and applications decide to begin storing complex matrices as two real matrices (traditionally-stored, by rows or columns), one each for real and imaginary components, the 2M approach (for numerically sensitive settings) as well as low-level applications of 3M (for numerically insensitive settings) become more appropriate.

6. Conclusions. We began the article by reviewing the general motivations for induced methods for complex matrix multiplication as well as the specific methods, 3M and 4M, studied in the previous article. Then, we recast complex scalar multiplication (and accumulation) in such a way that revealed a template that could be used to fashion a new induced method, one that casts complex matrix multiplication in terms of a single real matrix product. The key is the application of two new packing formats on the left- and right-hand matrix product operands that allows us to disguise the complex matrix multiplication as a real matrix multiplication with slightly modified input parameters. This 1M method is shown to have two variants, depending on whether the output matrix is stored by rows or columns. We also briefly contemplated a related 2M method that would be applicable to more exotic storage arrangements that separate the real and imaginary components of the matrix operands. When implemented in the BLIS framework, competitive performance was observed for 1M algorithms on a recent Intel microarchitecture. Finally, we reviewed the limitations of the 4M method that are overcome by 1M and concluded by discussing a few high-level observations.

[24] While this bandwidth distinction actually holds for all induced methods, different algorithms will place differing degrees of bandwidth pressure on the memory hierarchy, depending on the level(s) of cache from which they reuse F.E. of A and B.

[25] The bandwidth from the 1R-formatted matrix is unaffected since it only reorders (rather than duplicates) its F.E.. And since 1M uses half the k_C that is optimal in the real domain (and therefore would perform roughly twice as many rank-k_C updates), the bandwidth required for accessing F.E. of the output matrix may also be higher, though the precise level of increase will depend on the value of k_C^z that would be used by a comparable assembly-based implementation. However, bandwidth requirements on C are already quite low, so we would not expect this difference to measurably impact performance.

The key takeaway from our study of induced methods is that the real and imaginary elements of complex matrices can always be reordered to accommodate the desired fundamental primitives, whether those primitives are defined to be various forms of real matrix multiplication (as is the case for the 4M, 3M, 2M, and 1M methods), or vector instructions (as is the case for micro-kernels that implement complex arithmetic in assembly code). Indeed, even in the real domain, the classic matrix multiplication algorithm's packing format is simply a reordering of data that targets the fundamental primitive implicit in the micro-kernel—namely, an $m_R \times n_R$ rank-1 update. The family of induced methods presented here and in the previous article expand upon this basic reordering so that the mathematics of complex arithmetic can be expressed at different levels of the algorithm and of its corresponding implementation, each yielding different benefits, costs, and performance.

Appendix A. Additional Performance Results.

In this section we present performance results for implementations of 1M method on a recent ARM-based architecture. The primary purpose of gathering these results was to confirm 1M performance on an non-x86_64 architecture, and also to assess the relative performance of 1M on a platform for which Intel's MKL was not available.

A.1. Platform and implementation details. Results presented in this section were gathered on a single compute node consisting of two 28-core Marvell ThunderX2 CN9975 processors.[26] Each core, running at a clock rate of 2.2 GHz, provides a single-core peak performance of 17.6 gigaflops (GFLOPS) in double precision and 35.2 GFLOPS in single precision. Each socket has a 32MB L3 cache that is shared among cores, and each core has a private 256KB L2 cache and 32KB L1 (data) cache. Performance experiments were gathered under the Ubuntu 16.04 operating system running the Linux 4.15.0 kernel. Source code was compiled by the GNU C compiler (`gcc`) version 7.3.0.[27] The version of BLIS used in these tests was version 0.5.0-1.[28]

In this section, we show results for only 1M Algorithms 1M_C_BP, omitting results for the other three algorithms. We chose to omit 1M_R_BP because we did not develop a row-preferential micro-kernel for this architecture. (Recall that 1M_R variant algorithms require a micro-kernel that reads and writes elements of C in a row orientation.) And unlike the results shown in Section 4, we did not develop conventional assembly-based micro-kernels with which to compare. For reference, we also measured performance for the complex GEMM implementations found in OpenBLAS[29] and ARMPL 18.4.0.

All other parameters, such as values of α and β, and the number of trials performed for each problem size, as well as graphing conventions, such as scaling of the y-axis, remain identical to those of Section 4.

A.2. Analysis. Figure A.1 contains single-threaded (left) and multithreaded (right) performance of single-precision (top) and double-precision (bottom) complex

[26] While four-way symmetric multithreading is available on this hardware, the feature was disabled at boot-time so that the operating system detects only one logical core per physical core and schedules threads accordingly.

[27] The following optimization flags were used during compilation of BLIS and its test drivers: `-O3 -ftree-vectorize -mtune=cortex-a57`. In addition to those flags, the following flags were also used when compiling assembly kernels: `-march=armv8-a+fp+simd -mcpu=cortex-a57`.

[28] This version of BLIS may be uniquely identified, with high probability, by the first 10 digits of its `git` "commit" (SHA1 hash) number: e90e7f309b.

[29] This version of OpenBLAS may be uniquely identified, with high probability, by the first 10 digits of its `git` commit number: `52d3f7af50`.

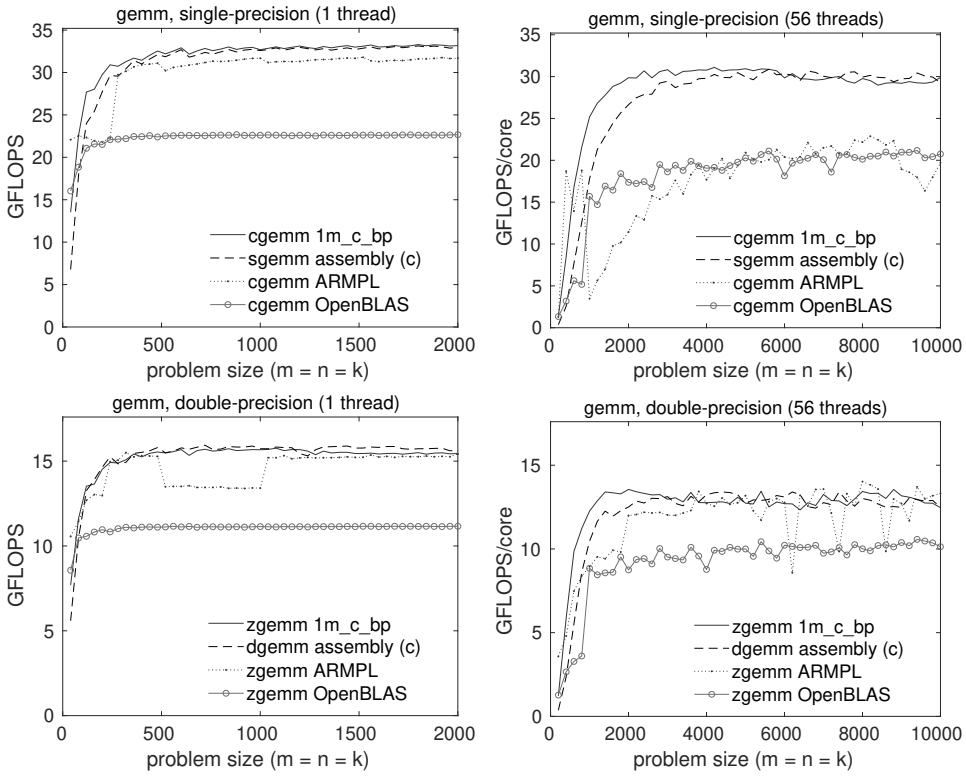

FIG. A.1. *Single-threaded (left) and multithreaded (right) performance of various implementations of single-precision (top) and double-precision (bottom) complex* GEMM *on a single core (left) or 56 cores (right) of a Marvell ThunderX2 CN9975 processor. All multithreaded data points reflect the use of 56 threads. The real domain* GEMM *implementation from BLIS uses a column-preferential microkernel, as indicated the a "(c)" in the legends. (The* 1M_C_BP *implementation uses the same column-preferential microkernel as the real domain* GEMM *implementation.) The theoretical peak performance coincides with the top of each graph.*

GEMM implementations. In addition to the 1M_C_BP implementation within BLIS, we also show the corresponding real domain GEMM implementation and the `cgemm` or `zgemm` found in OpenBLAS and ARMPL.

In Figure A.1 (top-left), single-precision 1M and its corresponding real domain benchmark track each other closely in the multithreaded configurations tested, as we would have expected. Somewhat surprisingly, the vendor library, ARMPL, does not appear to scale well at 56 threads (Figure A.1 (top-right)). Also somewhat surprisingly, OpenBLAS performance is consistently low, even for sequential execution. This suggests that while parallelism may be well-configured, their kernel is likely underperforming.

Figure A.1 (bottom) tells a similar story of performance among double-precision implementations, except that all BLIS implementations are, for reasons not immediately obvious, somewhat less efficient relative to peak performance than their single-precision counterparts. ARMPL performance is more competitive for both one and 56 threads, though the single-core graph exposes evidence of a "crossover point" strategy gone awry. ARMPL also seems to exhibit large swings in performance for certain large, multithreaded problem sizes. Once again, OpenBLAS performance is much

lower, but consistently so.

In summary, BLIS's 1M implementation performs extremely well on the Marvell CN9975 when computing in single precision. Performance and scalability in double precision, while not quite as impressive, is still highly competitive, especially when compared to OpenBLAS and the ARM Performance Library.

Acknowledgements. We kindly thank Devangi Parikh for gathering the ThunderX2 performance results presented in Appendix A. We also thank the Texas Advanced Computing Center for providing access to the the Intel Xeon "Lonestar5" compute cluster on which the performance data presented in Section 4 was gathered.

REFERENCES

[1] J. Dongarra, S. Hammarling, N. J. Higham, S. D. Relton, P. Valero-Lara, and M. Zounon, *The design and performance of batched BLAS on modern high-performance computing systems*, Procedia Computer Science, 108 (2017), pp. 495–504, https://doi.org/https://doi.org/10.1016/j.procs.2017.05.138, http://www.sciencedirect.com/science/article/pii/S1877050917307056.

[2] J. J. Dongarra, J. Du Croz, S. Hammarling, and I. Duff, *A set of level 3 basic linear algebra subprograms*, ACM Trans. Math. Soft., 16 (1990), pp. 1–17.

[3] G. Frison, D. Kouzoupis, T. Sartor, A. Zanelli, and M. Diehl, *BLASFEO: Basic linear algebra subroutines for embedded optimization*, ACM Trans. Math. Soft., 44 (2018), pp. 42:1–42:30, https://doi.org/10.1145/3210754, http://doi.acm.org/10.1145/3210754.

[4] K. Goto and R. A. van de Geijn, *Anatomy of high-performance matrix multiplication*, ACM Trans. Math. Soft., 34 (2008), pp. 12:1–12:25, https://doi.org/10.1145/1356052.1356053, http://doi.acm.org/10.1145/1356052.1356053.

[5] K. Goto and R. A. van de Geijn, *High-performance implementation of the level-3 BLAS*, ACM Trans. Math. Soft., 35 (2008), pp. 4:1–4:14, https://doi.org/10.1145/1377603.1377607, http://doi.acm.org/10.1145/1377603.1377607.

[6] J. A. Gunnels, G. M. Henry, and R. A. van de Geijn, *A family of high-performance matrix multiplication algorithms*, in Proceedings of the International Conference on Computational Sciences-Part I, ICCS '01, Berlin, Heidelberg, 2001, Springer-Verlag, pp. 51–60, http://dl.acm.org/citation.cfm?id=645455.653765.

[7] N. J. Higham, *Stability of a method for multiplying complex matrices with three real matrix multiplications*, SIAM J. Matrix Anal. App., 13 (1992), pp. 681–687, https://doi.org/10.1137/0613043, https://doi.org/10.1137/0613043.

[8] J. Huang, *Practical fast matrix multiplication algorithms*, (2018). PhD thesis, The University of Texas at Austin.

[9] J. Huang, D. A. Matthews, and R. A. van de Geijn, *Strassen's algorithm for tensor contraction*, SIAM Journal on Scientific Computing, 40 (2018), pp. C305–C326, https://doi.org/10.1137/17M1135578, https://doi.org/10.1137/17M1135578, https://arxiv.org/abs/https://doi.org/10.1137/17M1135578.

[10] J. Huang, L. Rice, D. A. Matthews, and R. A. van de Geijn, *Generating families of practical fast matrix multiplication algorithms*, in 31th IEEE International Parallel and Distributed Processing Symposium (IPDPS 2017), May 2017, pp. 656–667, https://doi.org/10.1109/IPDPS.2017.56.

[11] J. Huang, T. M. Smith, G. M. Henry, and R. A. van de Geijn, *Strassen's algorithm reloaded*, in Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '16, Piscataway, NJ, USA, 2016, IEEE Press, pp. 59:1–59:12, http://dl.acm.org/citation.cfm?id=3014904.3014983.

[12] J. Huang, C. D. Yu, and R. A. van de Geijn, *Implementing Strassen's algorithm with CUTLASS on NVIDIA Volta GPUs*, FLAME Working Note #88, TR-18-08, The University of Texas at Austin, Department of Computer Science, 2018, https://apps.cs.utexas.edu/apps/sites/default/files/tech_reports/GPUStrassen.pdf.

[13] Intel, *Math Kernel Library*. https://software.intel.com/en-us/mkl, 2019.

[14] Intel Corporation, *Intel® 64 and IA-32 Architectures Optimization Reference Manual*, no. 248966-033, June 2016.

[15] Intel Corporation, *Intel® Xeon® Processor E5 v3 Product Family: Processor Specification Update*, no. 330785-010US, September 2016.

[16] T. M. Low, F. D. Igual, T. M. Smith, and E. S. Quintana-Ortí, *Analytical modeling is*

enough for high-performance BLIS, ACM Trans. Math. Soft., 43 (2016), pp. 12:1–12:18, https://doi.org/10.1145/2925987, http://doi.acm.org/10.1145/2925987.

[17] *OpenBLAS*. http://xianyi.github.com/OpenBLAS/, 2019.

[18] T. M. Smith, R. A. van de Geijn, M. Smelyanskiy, J. R. Hammond, and F. G. Van Zee, *Anatomy of high-performance many-threaded matrix multiplication*, in Proceedings of the 28th IEEE International Parallel & Distributed Processing Symposium (IPDPS), IPDPS '14, Washington, DC, USA, 2014, IEEE Computer Society, pp. 1049–1059, https://doi.org/10.1109/IPDPS.2014.110, https://doi.org/10.1109/IPDPS.2014.110.

[19] F. G. Van Zee, *Inducing complex matrix multiplication via the 1m method*, FLAME Working Note #85 TR-17-03, The University of Texas at Austin, Department of Computer Sciences, 2017.

[20] F. G. Van Zee, T. Smith, F. D. Igual, M. Smelyanskiy, X. Zhang, M. Kistler, V. Austel, J. Gunnels, T. M. Low, B. Marker, L. Killough, and R. A. van de Geijn, *The BLIS framework: Experiments in portability*, ACM Trans. Math. Soft., 42 (2016), pp. 12:1–12:19, http://doi.acm.org/10.1145/2755561.

[21] F. G. Van Zee and T. M. Smith, *Implementing high-performance complex matrix multiplication via the 3m and 4m methods*, ACM Trans. Math. Soft., 44 (2017), pp. 7:1–7:36.

[22] F. G. Van Zee and R. A. van de Geijn, *BLIS: A framework for rapidly instantiating BLAS functionality*, ACM Trans. Math. Soft., 41 (2015), pp. 14:1–14:33, http://doi.acm.org/10.1145/2764454.

[23] R. C. Whaley, A. Petitet, and J. J. Dongarra, *Automated empirical optimization of software and the ATLAS project*, Parallel Computing, 27 (2001), pp. 3–35, https://doi.org/https://doi.org/10.1016/S0167-8191(00)00087-9, http://www.sciencedirect.com/science/article/pii/S0167819100000879. New Trends in High Performance Computing.

[24] C. D. Yu, J. Huang, W. Austin, B. Xiao, and G. Biros, *Performance optimization for the k-nearest neighbors kernel on x86 architectures*, in Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '15, New York, NY, USA, 2015, ACM, pp. 7:1–7:12, https://doi.org/10.1145/2807591.2807601, http://doi.acm.org/10.1145/2807591.2807601.